Design Phase Document

Robotics Competition Management System

Z. Kootbally[a]

[a]*MAGE, University of Maryland, College Park*

Abstract—This document captures the output of the design phase for the Robotics Competition Management System, a software tool for competition organizers and team managers. The system tracks team rosters, monitors robot status and battery levels, assigns and schedules tasks, models the arena layout, and records scoring. This document presents the domain model derived from noun/verb analysis, detailed class specifications with integrated business rules, structured behavioral scenarios with UML diagram references, and a phased implementation plan. The system is implemented across Lectures 6 and 7 of ENPM605.

Keywords—*OOP, design phase, robotics competition, Python, class design, UML, business rules, requirement analysis*

Contents

1 System Overview **2**

2 Domain Model **2**

2.1 Source Text . . . 2

2.2 Extraction Results . . . 2

2.3 Class Relationships . . . 3

3 Class Specifications **3**

3.1 Robot . . . 3
Attributes • Methods • Business Rules Enforced

3.2 Sensor . . . 5
Attributes • Methods • Business Rules Enforced

3.3 Task . . . 5
Attributes • Methods • Business Rules Enforced

3.4 Arena . . . 6
Attributes • Methods • Business Rules Enforced

3.5 Team . . . 6
Attributes • Methods • Business Rules Enforced

4 Behavioral Scenarios **7**

4.1 UC-01: Assign a Task to a Robot . . . 7

4.2 UC-02: Robot Completes a Task . . . 7

4.3 UC-03: Robot Navigates the Arena (L7) . . . 9

4.4 Additional Use Cases . . . 9

5 Implementation Plan **10**

5.1 Project Structure . . . 10

5.2 Phased Implementation . . . 10

5.3 Business Rules Traceability . . . 10

6 Glossary 11

1. System Overview

The Robotics Competition Management System is a software tool for competition organizers and team managers. It tracks team rosters, monitors robot status and battery levels, assigns and schedules tasks, models the arena layout, and records scoring. It does not implement robot control software (motor control, path planning, sensor fusion); it models the competition from the management perspective.

The system manages five core entities: robots, sensors, tasks, arenas, and teams. Robots are equipped with sensors and organized into teams. Tasks are assigned to individual robots, executed in an arena, and scored upon completion. The system enforces constraints on data validity (e.g., battery ranges, roster limits), triggers automatic state changes (e.g., low-battery alerts), and computes derived values (e.g., team scores).

This document serves as the bridge between the problem domain and the implementation phase, providing the blueprint from which Python code will be written in Lectures 6 and 7. Table 1 provides a quick reference for which classes and OOP concepts are covered in each lecture.

Table 1. Class-to-lecture mapping and OOP concepts introduced.

Lecture	Classes	OOP Concepts
L6	Robot, Sensor	Classes, objects, `__init__`, `self`, instance/-class attributes, dunder methods, encapsulation, `@property`
L7	Task, Team, Arena	Inheritance, polymorphism, abstract base classes, composition, aggregation, data classes

2. Domain Model

The domain model was derived from a noun/verb analysis of the competition description. Nouns in the description were identified as candidate classes or attributes, verbs as candidate methods, and possessive or relational phrases ("has a", "belongs to") as inter-class relationships.

2.1. Source Text

The following domain description was analyzed (nouns that became classes are shown in **bold**, candidate methods are shown in *italics*):

> A robotics competition involves **teams** of **robots** collaborating to *complete* **tasks** in an **arena**. Each **robot** is equipped with **sensors** to *perceive* its environment and must *navigate*, *pick up* objects, and *deliver* them to target zones. Teams *register* robots, *assign* tasks based on priority, and *track* scores as tasks are completed. Each robot has a battery level that *drains* during task execution and can be *recharged* to full capacity.

2.2. Extraction Results

The noun/verb analysis produced the following candidate classes. Note that this is a first-pass extraction; the class specifications in Section 3 refine these candidates by adding validation methods, computed properties, and implementation-specific signatures.

- **Robot**
  - *Attributes:* name, battery, status, tasks_completed, sensors
  - *Methods:* move(), perform_task(), recharge(), assign_task()
  - *Relationships:* Has Sensors, belongs to Team, assigned Tasks
- **Sensor**
  - *Attributes:* sensor_type, range_m, accuracy
  - *Methods:* read(), calibrate()
  - *Relationships:* Belongs to Robot
- **Task**
  - *Attributes:* name, priority, duration_s, points, status
  - *Methods:* assign(), complete(), cancel(), is_complete()
  - *Relationships:* Assigned to Robot
- **Arena**
  - *Attributes:* name, width, height, obstacles, zones
  - *Methods:* is_clear(), get_zone(), add_obstacle()
  - *Relationships:* Contains Robots
- **Team**
  - *Attributes:* team_name, robots, max_robots, score
  - *Methods:* add_robot(), remove_robot(), total_battery(), get_available_robots(), update_score()
  - *Relationships:* Has Robots

"Competition" and "zones" were considered during analysis and excluded. "Competition" may become a top-level orchestrating class in a future iteration. "Zones" are modeled as attributes of Arena.

2.3. Class Relationships

The following relationships were identified between classes. These directly inform how classes reference one another in the implementation.

- **Team to Robot (Aggregation):** A team manages a roster of robots. Robots can exist independently if the team is dissolved.
- **Robot to Sensor (Composition):** Sensors are owned by a robot. Destroying a robot destroys its sensors.
- **Task to Robot (Association):** A task is assigned to a robot. Both exist independently; the assignment can be broken.
- **Arena to Robot (Association):** The arena contains robots during competition. Neither owns the other.
- **MobileRobot/ManipulatorRobot to Robot (Inheritance, L7):** Specialized robot types extend the base Robot class.

3. Class Specifications

Each class specification includes its attributes with types and constraints, its methods with signatures and preconditions, and the business rules it enforces. Business rules are identified with a prefix: **BR-C** for constraints, **BR-T** for triggers, and **BR-D** for computations. Type annotations follow Python typing conventions.

Implementation details such as private helper methods and internal validation routines are deferred to the implementation lectures (L6 and L7). This section focuses on the public interface and the rules each class must enforce.

3.1. Robot

The central entity. Represents an autonomous agent registered to a team.

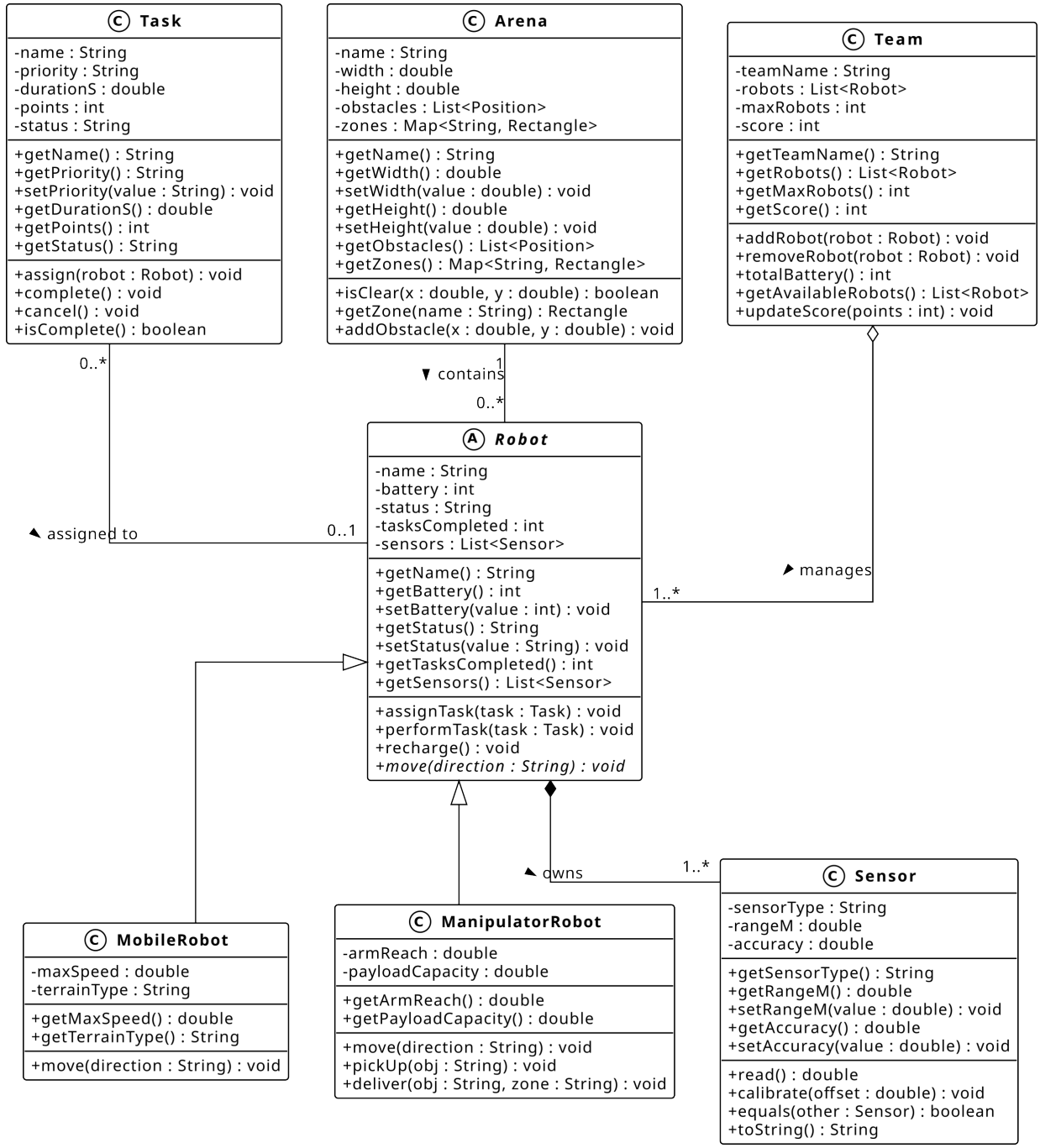

Figure 1. UML class diagram for the Robotics Competition Management System showing all five classes with attributes, methods, and relationships.

3.1.1. Attributes

- **name** (`str`): Required. Read-only after creation.
- **battery** (`int`): Must be in range [0, 100]. Default: 100. Validated in the property setter.
- **status** (`str`): One of: `'active'`, `'busy'`, `'needs_recharge'`, `'offline'`. Default: `'active'`.
- **tasks_completed** (`int`): Non-negative. Incremented on task completion. Default: 0.
- **sensors** (`list[Sensor]`): Composition. Sensors are created with the robot or added after.

3.1.2. Methods

- **assign_task**`(task: Task) -> None`: Status must be `'active'`. Raises `RuntimeError` otherwise. Sets status to `'busy'`.
- **perform_task**`(task: Task) -> None`: Status must be `'busy'`. Drains battery by `task.duration_s * drain_rate`. Evaluates trigger rules after draining.
- **recharge**`() -> None`: Sets battery to 100. Sets status to `'active'`.
- **move**`(direction: str) -> None`: Status must be `'active'` or `'busy'`.

3.1.3. Business Rules Enforced

- **BR-C01:** Battery must be an integer in [0, 100]. Enforced in the `battery` property setter.
- **BR-C02:** A robot cannot accept a task unless its status is `'active'`. Enforced as a precondition in `assign_task()`.
- **BR-T01:** When battery drops below 10, status automatically changes to `'needs_recharge'`. Evaluated after any operation that modifies battery.
- **BR-T03:** When battery reaches 0, status changes to `'offline'`. Evaluated after any operation that modifies battery.
- **BR-D01:** Battery drain = `task.duration_s * drain_rate`. Computed in `perform_task()`.

3.2. Sensor

Perception hardware mounted on a robot. Compared by range or accuracy.

3.2.1. Attributes

- **sensor_type** (`str`): One of: `'lidar'`, `'camera'`, `'ultrasonic'`. Read-only after creation.
- **range_m** (`float`): Must be positive. Validated in the property setter.
- **accuracy** (`float`): Must be in [0.0, 1.0]. Validated in the property setter.

3.2.2. Methods

- **read**`() -> float`: Returns `range_m` with simulated noise based on accuracy.
- **calibrate**`(offset: float) -> None`: Adjusts `range_m` by offset. Result must remain positive.
- **__str__**`() -> str`: Returns `'Sensor(<type>): range=<range_m>m'`.
- **__repr__**`() -> str`: Returns `"Sensor('<type>', <range_m>)"`.
- **__eq__**`(other: Sensor) -> bool`: Two sensors are equal if their `range_m` values are equal.
- **__gt__**`(other: Sensor) -> bool`: Compares sensors by `range_m`.
- **__add__**`(other: Sensor) -> Sensor`: Returns a new Sensor with type `'fused'` and summed ranges.

3.2.3. Business Rules Enforced

- **BR-C06:** Sensor range must be positive. Enforced in the `range_m` property setter.
- **BR-C07:** Accuracy must be in [0.0, 1.0]. Enforced in the `accuracy` property setter.

3.3. Task

A unit of work assigned to a robot. Scored upon completion.

3.3.1. Attributes

- **name** (`str`): Required. Describes the task (e.g., "pick widget from zone A").
- **priority** (`str`): Must be one of: `'low'`, `'medium'`, `'high'`, `'critical'`. Validated in the property setter.
- **duration_s** (`float`): Must be positive. Estimated execution time in seconds.
- **points** (`int`): Must be non-negative. Points awarded on completion.
- **status** (`str`): One of: `'pending'`, `'in_progress'`, `'completed'`, `'cancelled'`. Default: `'pending'`.

3.3.2. Methods

- **assign**`(robot: Robot) -> None`: Status must be `'pending'`. Sets status to `'in_progress'`. Calls `robot.assign_task(self)`.
- **complete**`() -> None`: Status must be `'in_progress'`. Sets status to `'completed'`. Triggers team score update (BR-T02).
- **cancel**`() -> None`: Status must be `'pending'` or `'in_progress'`. Sets status to `'cancelled'`. Robot (if assigned) returns to `'active'`.
- **is_complete**`() -> bool`: Returns `True` if status == `'completed'`.

3.3.3. Business Rules Enforced

- **BR-C05:** Priority must be one of the four allowed values. Enforced in the `priority` property setter.
- **BR-T02:** On task completion, team score is updated by adding `task.points`. Triggered in `complete()`.

3.4. Arena

The competition environment. Owns obstacle data and zone definitions. Arena behavior is primarily exercised in L7 scenarios involving robot navigation and task execution within the physical space.

3.4.1. Attributes

- **name** (`str`): Required.
- **width** (`float`): Must be positive. Validated in the property setter.
- **height** (`float`): Must be positive. Validated in the property setter.
- **obstacles** (`list[tuple]`): List of (x, y) positions. Default: empty.
- **zones** (`dict[str, tuple]`): Named zones mapping to (x, y, w, h) rectangles. Default: empty.

3.4.2. Methods

- **is_clear**`(x: float, y: float) -> bool`: Returns `True` if position does not overlap any obstacle.
- **get_zone**`(name: str) -> tuple`: Returns the bounding rectangle for the named zone. Raises `KeyError` if not found.
- **add_obstacle**`(x: float, y: float) -> None`: Appends (x, y) to the obstacle list.

3.4.3. Business Rules Enforced

- **BR-C08:** Width and height must be positive. Enforced in property setters.

3.5. Team

A group of robots registered to compete. Manages roster and scoring.

3.5.1. Attributes

- **team_name** (`str`): Required. Read-only after creation.
- **robots** (`list[Robot]`): Managed via add/remove methods, not directly.
- **max_robots** (`int`): Must be positive. Maximum roster size. Default: 5.
- **score** (`int`): Non-negative. Updated on task completion. Default: 0.

3.5.2. Methods

- **add_robot**`(robot: Robot) -> None`: Roster must not exceed `max_robots` (BR-C04). Robot must not belong to another team (BR-C03). Raises `RuntimeError` on violation.
- **remove_robot**`(robot: Robot) -> None`: Robot must be in the roster. Raises `ValueError` otherwise.
- **total_battery**`() -> int`: Returns sum of battery across all robots (BR-D03).
- **get_available_robots**`() -> list[Robot]`: Returns robots with status == `'active'`.
- **update_score**`(points: int) -> None`: Adds points to score. Called on task completion (BR-T02, BR-D02).

3.5.3. Business Rules Enforced

- **BR-C03:** Each robot belongs to exactly one team. Enforced in `add_robot()` by checking the robot's current team.
- **BR-C04:** Roster cannot exceed `max_robots`. Enforced as a precondition in `add_robot()`.
- **BR-D02:** `team.score` = sum of completed task points. Maintained incrementally via `update_score()`.
- **BR-D03:** `total_battery()` = sum of `robot.battery` for all robots. Computed on demand.

4. Behavioral Scenarios

Each scenario documents a use case with preconditions, the main flow, postconditions, and exception paths. The two primary scenarios include references to sequence and activity diagrams.

4.1. UC-01: Assign a Task to a Robot

- **Actors:** Team manager (implicit, via code).
- **Preconditions:** Team has at least one robot with status `'active'`. Task exists with status `'pending'`.
- **Main Flow:**
  1. Manager calls `team.get_available_robots()` to find eligible robots.
  2. For a candidate robot, the system checks `robot.status == 'active'`.
  3. The system verifies `robot.battery` >= estimated drain for the task.
  4. `task.assign(robot)` is called, setting `task.status` to `'in_progress'`.
  5. `robot.assign_task(task)` is called, setting `robot.status` to `'busy'`.
- **Postconditions:** Task status is `'in_progress'`. Robot status is `'busy'`. Task is linked to the robot.
- **Exceptions:**
  - 3a. No robots have status `'active'`: raise `RuntimeError`.
  - 3b. Robot battery insufficient: skip to next candidate or raise error.

4.2. UC-02: Robot Completes a Task

- **Actors:** Robot (autonomous execution).
- **Preconditions:** Robot status is `'busy'`. A task with status `'in_progress'` is assigned.
- **Main Flow:**
  1. Robot checks battery level against `task.duration_s * drain_rate`.
  2. If sufficient: robot navigates to the target zone.
  3. Robot picks up the object at the source zone.
  4. Robot delivers the object to the target zone.
  5. Robot calls `task.complete()`, setting status to `'completed'`.
  6. `task.complete()` triggers `team.update_score(task.points)`.
  7. Robot increments `tasks_completed` and sets status to `'active'`.
  8. Battery trigger rules are evaluated (BR-T01, BR-T03).

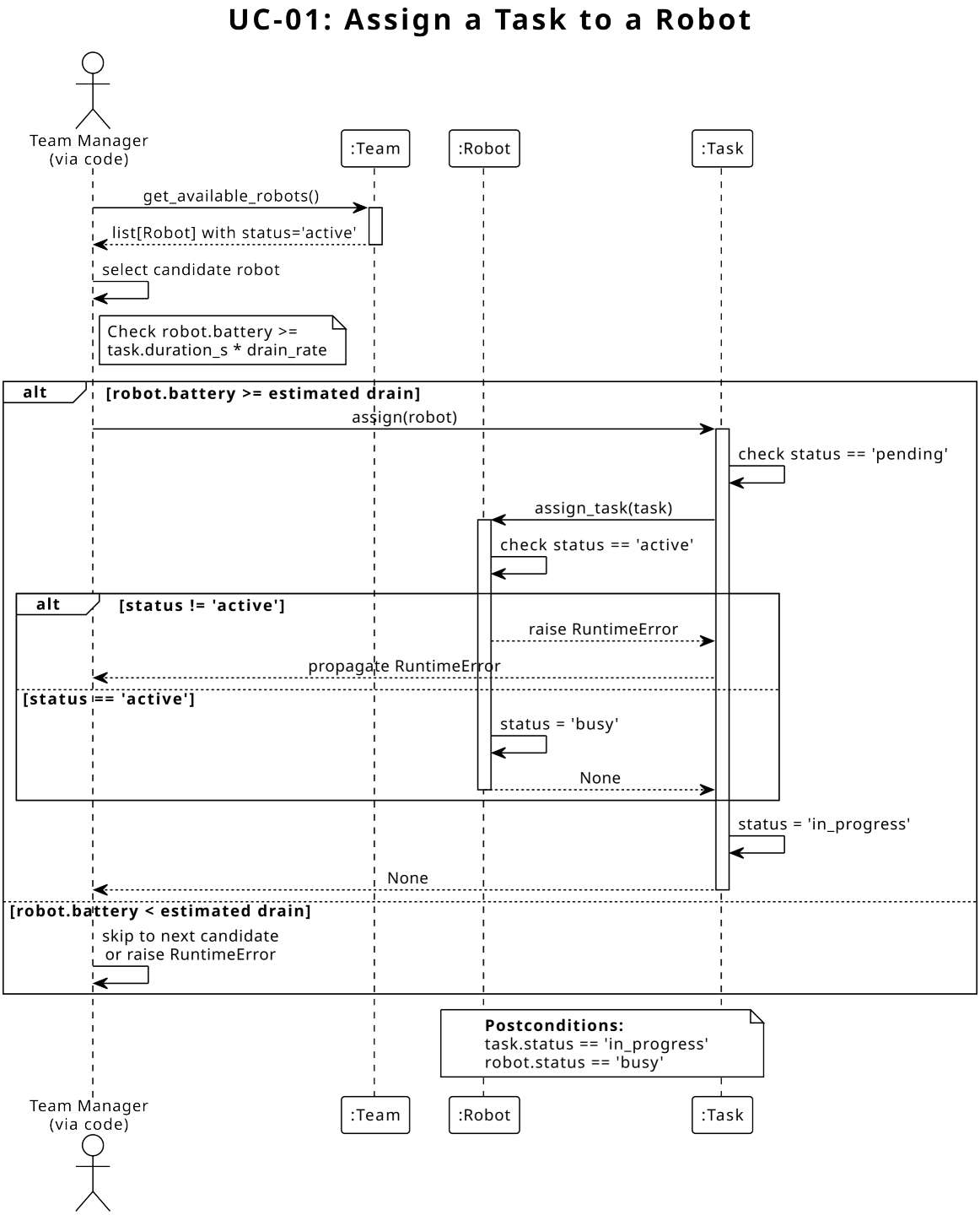

Figure 2. Sequence diagram for UC-01: Assign a Task to a Robot.

- **Postconditions:** Task status is `'completed'`. Team score increased. Robot status is `'active'` (or `'needs_recharge'` if battery < 10).
- **Exceptions:**
  - 1a. Battery insufficient: robot calls `recharge()` first, then re-checks.
  - 4a. Battery depleted mid-task: task is cancelled, robot status set to `'offline'`.

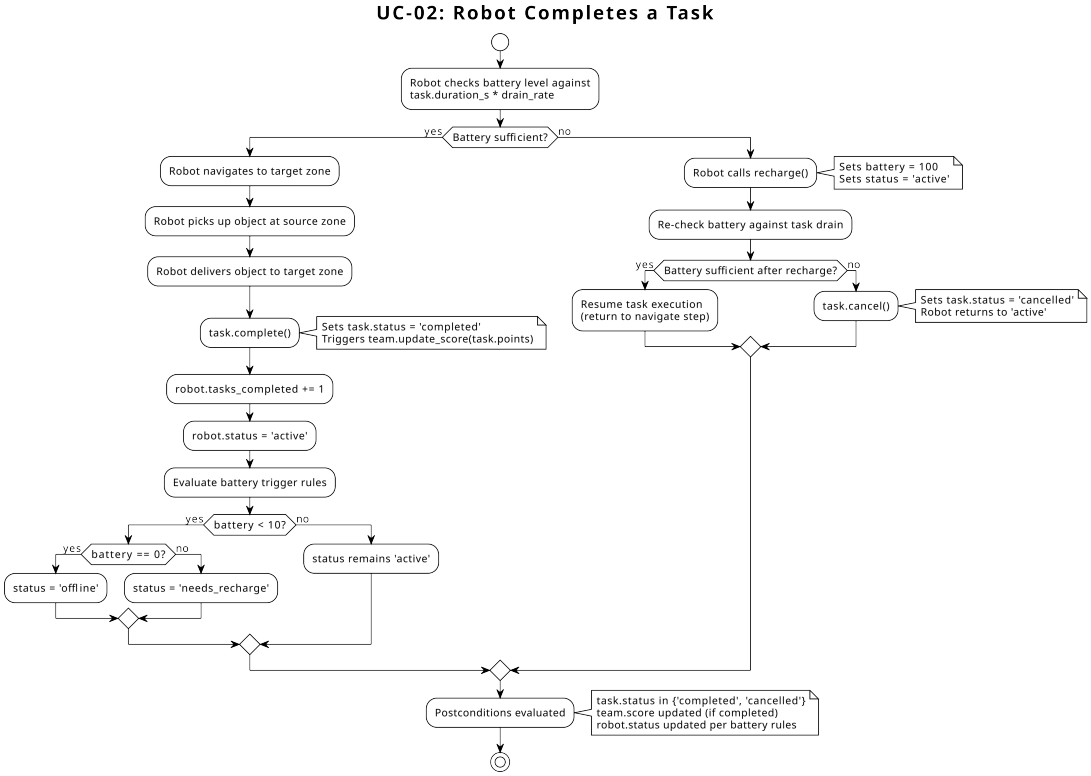

Figure 3. Activity diagram for UC-02: Robot Completes a Task.

4.3. UC-03: Robot Navigates the Arena (L7)

- **Actors:** Robot, Arena.
- **Preconditions:** Robot is assigned a task. Arena is initialized with obstacles and zones.
- **Main Flow:**
  1. Robot queries `arena.get_zone(target)` to obtain the target coordinates.
  2. Robot checks `arena.is_clear(x, y)` along its planned path.
  3. If the path is clear, robot moves to the target zone.
  4. If an obstacle is encountered, robot recalculates the path.
- **Postconditions:** Robot has reached the target zone or raised an error.
- **Exceptions:**
  - 1a. Zone not found: `KeyError` is raised.
  - 2a. No clear path exists: robot reports failure.

4.4. Additional Use Cases

- **UC-04: Register a Team.** Create a Team with a name and `max_robots`. Postcondition: empty roster, score $= 0$.
- **UC-05: Add Robot to Team.** Call `team.add_robot(robot)`. Enforces BR-C03 (unique team) and BR-C04 (roster limit).

- **UC-06: Equip Robot with Sensor.** Create Sensor and append to `robot.sensors`. Enforces BR-C06 and BR-C07.
- **UC-07: Recharge a Robot.** Call `robot.recharge()`. Sets battery to 100, status to `'active'`.
- **UC-08: Cancel a Task.** Call `task.cancel()`. Status must be `'pending'` or `'in_progress'`. Robot (if assigned) returns to `'active'`.
- **UC-09: Query Team Standings.** Call `team.score` and `team.total_battery()`. Returns current score and aggregate battery.

5. Implementation Plan

5.1. Project Structure

Each class lives in its own module. This promotes modularity and makes the code easier to maintain, test, and extend.

- `robot.py`: Robot class (L6)
- `sensor.py`: Sensor class (L6)
- `task.py`: Task class (L7)
- `team.py`: Team class (L7)
- `arena.py`: Arena class (L7)
- `main.py`: Entry point (L6 / L7)

5.2. Phased Implementation

Table 1 (Section 1) summarizes the mapping. The two phases are:

- **L6: Core Classes.** Robot and Sensor with full validation, dunder methods, and `@property`. Students implement classes from the specifications in Sections 3.1 and 3.2, learning encapsulation and operator overloading.
- **L7: Relationships and Specialization.** Task, Team, and Arena with inheritance (`MobileRobot`, `ManipulatorRobot`) and abstract Task interface. Students implement composition, aggregation, and polymorphism using the specifications in Sections 3.3 through 3.5.

5.3. Business Rules Traceability

Every business rule identified in the class specifications maps to a specific enforcement point in the code.

Constraints:

- **BR-C01:** Battery in [0, 100]. Enforced in the `Robot.battery` property setter.
- **BR-C02:** Robot must be `'active'` to accept tasks. Enforced in `Robot.assign_task()` precondition.
- **BR-C03:** Robot belongs to exactly one team. Enforced in `Team.add_robot()` check.
- **BR-C04:** Roster cannot exceed `max_robots`. Enforced in `Team.add_robot()` precondition.
- **BR-C05:** Priority must be low/medium/high/critical. Enforced in the `Task.priority` property setter.
- **BR-C06:** Sensor range must be positive. Enforced in the `Sensor.range_m` property setter.
- **BR-C07:** Accuracy in [0.0, 1.0]. Enforced in the `Sensor.accuracy` property setter.
- **BR-C08:** Arena dimensions must be positive. Enforced in the `Arena` width/height property setters.

Triggers:

- **BR-T01:** Battery < 10 triggers `'needs_recharge'`. Evaluated after any operation that modifies battery.
- **BR-T02:** Task completion triggers score update. Enforced in `Task.complete()` which calls `team.update_score`
- **BR-T03:** Battery $== 0$ triggers `'offline'`. Evaluated after any operation that modifies battery.

Computations:

- **BR-D01:** Drain = duration $\times$ rate. Computed in `Robot.perform_task()`.
- **BR-D02:** Score = sum of task points. Maintained incrementally via `Team.update_score()`.

- **BR-D03:** Total battery = sum across roster. Computed on demand in `Team.total_battery()`.

6. Glossary

- **Aggregation:** A "has-a" relationship where the part can exist independently of the whole (e.g., Team has Robots).
- **Association:** A general relationship between two classes where neither owns the other (e.g., Task is assigned to Robot).
- **Business Rule:** A constraint, trigger, or computation that the system must enforce as part of domain logic.
- **Composition:** A "has-a" relationship where the part cannot exist without the whole (e.g., Robot has Sensors).
- **Domain Model:** The set of classes, attributes, methods, and relationships that represent the problem space.
- **Drain Rate:** A constant factor determining how much battery is consumed per second of task execution.
- **Dunder Method:** A Python method with double underscores (e.g., `__init__`, `__str__`) enabling operator overloading and integration with built-in functions.
- **Encapsulation:** Bundling data and methods together while restricting direct access to internal state.
- **Noun/Verb Analysis:** A technique for extracting candidate classes (nouns) and methods (verbs) from a natural-language description.
- **Precondition:** A condition that must be true before a method can execute. Raises an exception if violated.
- **Property:** A Python mechanism (via the `@property` decorator) that allows controlled access to an attribute through getter and setter methods while preserving attribute-style syntax. Used throughout this system to enforce validation on assignment.
- **UML:** Unified Modeling Language. A standardized notation for visualizing software design.